NPTEL ETHICAL HACKING & NETWORK COMMUNICATIONS

WEEK 7 DETAILED CLASSROOM STUDY NOTES & EXAM PREPARATION NOTEBOOK

Course: NPTEL Ethical Hacking | Coverage: Lectures 31 to 35 (Cryptographic Hash Functions, Digital Signatures, PKI & Certificates, SSL/TLS, SSH, IPsec & Security Applications)

MODULE 1: LECTURE 31 — CRYPTOGRAPHIC HASH FUNCTIONS (PART I)

1.1 Introduction & Foundational Concepts

Simple Explanation: Imagine you have a massive 1,000-page book. If you want to create a tiny 'digital fingerprint' of that book—a fixed 64-character code that uniquely represents the entire text—you run it through a cryptographic hash function. If even a single comma inside that 1,000-page book is changed, the entire fingerprint completely changes! However, you can never reconstruct the 1,000 pages back from that 64-character code.

Why It Is Needed: In internet communications, physical verification of identity and document integrity is impossible. We need a mathematical tool to verify that a file or message has not been tampered with in transit (Integrity) and to confirm who sent it (Authentication). Hash functions form the foundation of digital signatures, password storage, file verification, and secure protocols.

Technical Definition: A *Hash Function* is a mathematical algorithm H that maps an input message M of arbitrary bit length to a fixed-length output string $h = H(M)$, called the *hash digest*, *hash value*, or *fingerprint*. A *Cryptographic Hash Function* possesses specific security properties that make it computationally infeasible to reverse or manipulate.

CONCEPT FLOW: HASHING PROCESS (MANY-TO-ONE MAPPING)

Input Message (M) [Arbitrary Length: 10 bits or 10 GB]
↓ (Passes through Hash Algorithm H)
Hash Digest (h = H(M)) [Fixed Length: e.g., 128-bit MD5, 256-bit SHA-256, 512-bit SHA-512]

Key Structural Observation: Since the input space is infinite (2^{∞}) and the output space is finite (2^n), hashing is inherently a **many-to-one mathematical mapping**. Multiple distinct inputs can theoretically map to the same hash value (a **Collision**).

1.2 Classification: Unkeyed vs. Keyed Hash Functions

Hash functions in network security are categorized based on whether they require a shared secret key:

Category	Technical Name	Key Usage	Security Goal Provided
Unkeyed Hash Function	Modification Detection Code (MDC)	No secret key used.	**Message Integrity ONLY.** Detects accidental or malicious alterations in transit.
Keyed Hash Function	Message Authentication Code (MAC)	Uses a shared secret key K.	**Message Integrity AND Sender Authentication.** Proves data came from secret key owner.

Step-by-Step Mechanics of MDC (Unkeyed) for Integrity Verification:

1. **Sender:** Computes $MDC = H(M)$ and transmits $(M \| MDC)$ to the receiver over the network.
2. **Receiver:** Receives $(M' \| MDC')$. Separates received message M' from received hash MDC'.
3. **Computation:** Receiver computes $MDC_{calc} = H(M')$.
4. **Verification:** Compares MDC_{calc} with MDC'. If equal, data integrity is intact! If different, message was modified.

Note: Anyone can recompute an MDC, so MDC does NOT prove who sent the message.

1.3 Six Desirable Security Properties of Cryptographic Hash Functions

For a standard hash function to qualify as *cryptographically secure*, it must strictly satisfy the following six mathematical criteria:

- **1. Variable Input Size:** Can be applied to an input message M of virtually any length or size (from 1 byte to terabytes).
- **2. Fixed Output Length:** Always produces a hash digest h of a predetermined, fixed bit-length n (e.g., 128-bit for MD5, 256-bit for SHA-256).
- **3. Computational Efficiency:** Computing $h = H(M)$ is fast and easy for any given message M in both software and hardware.
- **4. First Preimage Resistance (One-Way Property):** Given a hash output y, it is computationally infeasible to find any original input x such that $H(x) = y$. You cannot reverse the hash!
- **5. Second Preimage Resistance (Weak Collision Resistance):** Given a specific input message x, it is computationally infeasible to find a *different* message $x' \neq x$ such that $H(x') = H(x)$. Prevents message substitution attacks.
- **6. Collision Resistance (Strong Collision Resistance):** It is computationally infeasible to find *any pair* of arbitrary distinct input messages (x, x') such that $x \neq x'$ and $H(x) = H(x')$. This is the strongest security property!

1.4 Cryptographic Hash Function Families

Over the history of cybersecurity, three primary families of hash functions have been deployed:

- **MD (Message Digest) Family:** Developed by Ronald Rivest. Includes MD2, MD4, and **MD5** (produces a 128-bit digest). *Exam Warning:* MD5 is now cryptographically broken due to practical collision vulnerabilities!
- **SHA (Secure Hash Algorithm) Family:** Published by NIST/NSA. Standard choice today.
  - **SHA-1:** Produces 160-bit digest (deprecated due to theoretical vulnerabilities).
  - **SHA-2:** Includes SHA-224, SHA-256, SHA-384, SHA-512 (produces 256-bit or 512-bit digests; current global standard).
  - **SHA-3 (Keccak):** Latest standard offering alternative structural design resistant to SHA-2 attack vectors.
- **RIPEMD Family:** Developed in Europe. Includes RIPEMD-128 and RIPEMD-160 (produces 160-bit digest, used in Bitcoin addresses).

EXAM POINT & COMMON CONFUSION
Question: Does an unkeyed hash function (MDC) provide non-repudiation or authentication?
Answer: NO! An unkeyed hash function provides *data integrity ONLY*. Because anyone can calculate $H(M)$, an attacker can alter both the message and the hash. Authentication and non-repudiation require a **shared secret key (MAC)** or a **private key (Digital Signature)**.

MODULE 2: LECTURE 32 — CRYPTOGRAPHIC HASH FUNCTIONS (PART II)

2.1 Practical Applications of One-Way Hash Functions

Cryptographic hash functions are versatile primitives used across several critical security workflows:

- **1. Secure Password Storage:** Instead of storing user passwords in plaintext (which creates catastrophic risk if the database is breached), systems store $H(\text{Password})$. During login, the system computes $H(\text{Entered Password})$ and compares it to the stored hash. Even if attackers steal the password table, they cannot reverse the hashes to obtain plaintext passwords!
- **2. Symmetric-Key Authentication & Integrity:** Sender hashes message M, encrypts hash with shared key $K \implies E_K(H(M))$, and transmits $(M || E_K(H(M)))$. Receiver decrypts hash using K and compares it to $H(M')$. Provides both authentication and integrity.
- **3. Digital Signatures (Public-Key Cryptography):** Sender hashes message M, encrypts hash using sender's **Private Key (d)** $\implies S = E_d(H(M))$. Receiver decrypts signature using sender's **Public Key (e)** and compares it to $H(M')$. Provides authentication, integrity, AND **Non-Repudiation**!
- **4. Secret Key Concatenation (No Encryption):** Sender appends shared secret key K to message M and hashes the concatenated string $\implies H(M || K)$. Sender transmits $(M || H(M || K))$ without sending K. Receiver appends K to M' and recomputes $H(M' || K)$. High speed because no encryption algorithm is executed!

Performance Comparison Hierarchy:

Cryptographic Operation	Relative Speed	Computational Overhead	Primary Use Case
Pure Hash Computation	Fastest	Very Low	Integrity & $H(M\|\|K)$ MACs
Symmetric Encryption	Moderate	Medium	Bulk data confidentiality
Public-Key Encryption	Slowest	Very High	Key exchange & Digital Signatures

2.2 SHA-512 Architectural Breakdown

SHA-512 is a key member of the SHA-2 family that processes input messages in **1024-bit blocks** and outputs a **512-bit digest**.

Step 1: Message Preprocessing & Padding:
• The message M of length L bits is padded so that its total length is a multiple of 1024 bits.
• Padding structure: Append a single '1' bit, followed by k '0' bits, followed by an unsigned 128-bit integer representing the original length L.
• Mathematical Condition: $(L + 1 + k) \equiv 896 \pmod{1024} \implies (L + 1 + k + 128) \equiv 0 \pmod{1024}$.

Step 2: Iterative Block Chaining:
• Padded message is divided into N blocks of 1024 bits: $M_1, M_2, \dots, M_N$.
• Uses an initial 512-bit constant value H_0 (derived from square roots of first 8 prime numbers).
• Recurrence Formula: $H_i = F(H_{i-1}, M_i)$ for $i = 1, 2, \dots, N$. Final digest is H_N.

Step 3: Compression Function F (80 Rounds):
• Each 1024-bit block M_i goes through **80 processing rounds**.
• Uses an 8-element 64-bit working register buffer (A, B, C, D, E, F, G, H) totaling 512 bits.
• Each round t ($0 \le t \le 79$) uses a 64-bit round constant K_t (derived from **fractional parts of cube roots of first 80 prime numbers**) and a 64-bit message word W_t derived from M_i.

2.3 HMAC (Hash-based Message Authentication Code)

What it is: HMAC is a standard construction (RFC 2104) for calculating a Message Authentication Code using an existing cryptographic hash function (like SHA-256 or SHA-512) combined with a secret key K.

Why it is needed: Simple secret key hashing like $H(K || M)$ is vulnerable to *Length Extension Attacks*. HMAC uses a two-pass nested hashing structure to eliminate this vulnerability without requiring slow encryption algorithms.

HMAC MATHEMATICAL FORMULA & ARCHITECTURE

$$\mathbf{HMAC(K, M) = H\Big((K \oplus opad) \;||\; H\big((K \oplus ipad) \;||\; M\big)\Big)}$$

Symbols & Definitions:
• H: Embedded cryptographic hash function (e.g., SHA-256 or SHA-512).
• K: Shared secret key (padded with zeros to match hash block size B, e.g., 1024 bits for SHA-512).
• M: Input message to be authenticated.
• $ipad$: Inner pad constant (byte `0x36` repeated $B/8$ times).
• $opad$: Outer pad constant (byte `0x5C` repeated $B/8$ times).
• $\oplus$: Bitwise Exclusive-OR (XOR) operation.
• $||$: String concatenation.

2.4 Attacks on Hash Functions & The Birthday Paradox

The primary threat against hash functions is finding a **Collision** ($x \neq x'$ where $H(x) = H(x')$).

The Birthday Attack: Based on the mathematical Birthday Paradox in probability theory (in a room of just 23 people, there is a $>50\%$ chance that two people share the exact same birthday!).

Mathematical Security Impact:
• For a hash function with an n-bit output digest, the total possible hash values is 2^n.
• A naive brute-force search to find a preimage for a *specific* hash takes 2^n operations.
• However, finding *any arbitrary pair* of colliding messages (x, x') requires only approximately **$2^{n/2}$ attempts**!
• *Key Conclusion:* The effective security strength of an n-bit hash function against collision attacks is **$n/2$ bits**. Thus, to achieve 64-bit security against collisions, n must be at least **128 bits** (MD5). To achieve 256-bit security, n must be **512 bits** (SHA-512)!

MODULE 3: LECTURE 33 — DIGITAL SIGNATURE AND CERTIFICATE

3.1 Principles of Digital Signatures

Simple Explanation: When you sign a physical paper check with a pen, your signature proves you authorized the payment. A *Digital Signature* is the mathematical equivalent for digital data—but far superior! It binds the signer's identity to the exact contents of the message. If anyone changes even a single bit of the document after it is signed, the signature instantly becomes invalid.

Three Core Security Objectives Provided:
1. **Sender Authentication:** Confirms the message originated from the specific private key owner.
2. **Message Integrity:** Guarantees the message contents were not modified in transit.
3. **Non-Repudiation:** The signer *cannot deny* having signed the message, because only they possess their secret private key!

STEP-BY-STEP DIGITAL SIGNATURE WORKFLOW

SIGNING PROCESS (Sender Alice):
1. Alice computes hash of message M: $m = H(M)$.
2. Alice encrypts/signs hash m using her **Private Key (d_A)**: $S = f_s(m, d_A)$.
3. Alice transmits $(M || S)$ to Bob.

VERIFICATION PROCESS (Receiver Bob):
1. Bob receives $(M' || S')$. Computes hash of received message: $m' = H(M')$.
2. Bob decrypts signature S' using Alice's **Public Key (e_A)**: $m'' = f_v(S', e_A)$.
3. Bob compares m' with m''. If $m' == m''$, signature is **VALID**!

3.2 Taxonomy & Types of Digital Signatures

- **Signature with Appendix (Standard):** The signature S is generated by signing the hash $H(M)$ and appended to M. Highly efficient because only a short digest is cryptographically processed. Standard in X.509 and SSL/TLS.
- **Signature with Message Recovery:** The entire message M is encrypted with the private key to form S. Receiver decrypts S using public key to recover M. Inefficient for large files; rarely used.
- **Deterministic vs. Probabilistic:** Deterministic signatures (e.g., standard RSA) always yield the exact same signature string for a given message and key. Probabilistic signatures (e.g., ElGamal, DSA) inject random numbers so the signature string differs on every signing attempt.
- **Blind Signatures:** Allows a user to get a message signed by an authority *without revealing the message content* to the authority! User multiplies message m by blinding factor $k^e \pmod n \implies m^*$. Authority signs m^*. User un-blinds the result to obtain valid signature S. Critical for **untraceable digital cash & e-voting**.
- **Undeniable Signatures:** Requires the active online cooperation of the signer to verify the signature. Prevents unauthorized third parties from verifying signatures without signer consent.

3.3 Attacks on Digital Signatures

Attackers attempt to compromise digital signatures across several severity levels:

1. **Total Break:** Attacker successfully determines or derives the signer's private key d. Attacker can now forge any signature on any message!
2. **Selective Forgery:** Attacker creates a valid signature for a *specific targeted message* chosen in advance by the attacker.
3. **Existential Forgery:** Attacker manages to create a valid signature for *some arbitrary/garbage message* that the attacker could not

control in advance.

3.4 Public Key Infrastructure (PKI) & Digital Certificates

The Public Key Distribution Problem: If Alice sends Bob her public key over the internet, how does Bob know it really belongs to Alice and not to an attacker (Eve) conducting a Man-in-the-Middle attack?

The Solution — Digital Certificates & CAs: A *Digital Certificate* is an electronic document issued by a trusted third party called a **Certification Authority (CA)** that officially binds an entity's identity to their public key.

- **X.509 v3 Standard Certificate Fields:** Structure defined by ITU-T / ISO X.509 standard:
- **Subject Name:** Identity of the certificate owner (e.g., `CN=amazon.com, O=Amazon Inc`).
- **Public Key:** The owner's public key value and algorithm identifier (e.g., RSA 2048-bit).
- **Issuer CA:** Name of the trusted CA that verified identity and issued certificate (e.g., `DigiCert`).
- **Validity Period:** Start Date (Not Before) and Expiration Date (Not After).
- **Serial Number:** Unique identifier assigned by CA for tracking and revocation.
- **CA Digital Signature:** The digital signature generated by the CA's private key over the certificate fields. Verified by browsers using pre-installed CA public keys!

3.5 Certificate Revocation & Elliptic Curve Cryptography (ECC)

Certificate Revocation List (CRL): If a private key is compromised, or organizational details change before expiration, the CA revokes the certificate and publishes its serial number on a publicly accessible **CRL**.

Elliptic Curve Cryptography (ECC): An alternative public-key architecture based on the algebraic structure of elliptic curves over finite fields. Offers equivalent security to RSA with **dramatically smaller key sizes** (e.g., 256-bit ECC key $\approx$ 3072-bit RSA key!). Ideal for resource-constrained IoT, mobile devices, and smart cards due to low memory, power, and bandwidth overhead.

MODULE 4: LECTURE 34 — APPLICATIONS (PART I): SSL ARCHITECTURE

4.1 Overview & Goals of Secure Socket Layer (SSL)

What it is: The Secure Socket Layer (SSL) protocol (pioneered by Netscape) sits as an optional security layer directly above TCP and below application protocols (like HTTP, LDAP, POP3).

Three Core Security Goals Provided:
1. **Server & Client Authentication:** Uses X.509 digital certificates to verify party identities and prevent impersonation.
2. **Data Confidentiality (Encryption):** Encrypts application payloads and control headers using symmetric ciphers (e.g., AES, 3DES).
3. **Data Integrity:** Appends Message Authentication Codes (MACs) using keyed hash functions to detect tampering.

4.2 SSL Two-Layer Protocol Architecture

SSL is logically structured into two distinct operational layers:

Layer Level	Protocol Name	Functional Description & Role
Lower Layer (Above TCP)	**SSL Record Protocol**	Handles data fragmentation (≤ 16 KB), compression, MAC generation, payload encryption, and header encapsulation before handing data to TCP.
Higher Layer (Above Record)	**SSL Handshake Protocol**	Most complex protocol. Negotiates cipher suites, exchanges keys, and authenticates client/server across 4 phases.

Higher Layer	**SSL ChangeCipherSpec Protocol**	Single 1-byte message (`0x01`) signaling immediate activation of negotiated pending security parameters.
Higher Layer	**SSL Alert Protocol**	Conveys 2-byte alert messages: Severity Byte (1=Warning, 2=Fatal Error) and Alert Code Byte. Fatal errors terminate session.

4.3 SSL Record Protocol Data Processing Pipeline

Every application payload or higher-layer SSL message passes through 6 sequential steps in the SSL Record Protocol:

1. **Fragmentation:** Application data stream is fragmented into blocks of maximum size **16 KB (16,384 bytes)**.
2. **Compression:** Each fragment is optionally compressed (must be lossless).
3. **MAC Generation:** A keyed MAC is calculated over the compressed data using a shared secret key.
4. **Encryption:** The compressed payload + MAC are encrypted using symmetric session key.
5. **SSL Header Addition:** Appends a 5-byte header containing *Content Type* (20=ChangeCipherSpec, 21=Alert, 22=Handshake, 23=Application), *Major/Minor Version*, and *Compressed Length*.
6. **Hand-off to TCP:** Transmitted over standard TCP connection.

4.4 Detailed 4-Phase SSL Handshake Protocol Workflow

FOUR-PHASE SSL HANDSHAKE LIFECYCLE

PHASE 1: Establish Security Capabilities
• Client sends `ClientHello` (SSL version, random number R_C, session ID, supported CipherSuites).
• Server responds with `ServerHello` (chosen SSL version, random number R_S, session ID, chosen CipherSuite).

PHASE 2: Server Authentication & Key Exchange
• Server sends `Certificate` (contains server's X.509 public key).
• Server sends `ServerKeyExchange` (optional, if certificate is insufficient for key exchange).
• Server sends `CertificateRequest` (optional, for client auth) & `ServerHelloDone`.

PHASE 3: Client Authentication & Key Exchange
• Client sends `Certificate` (if requested by server).
• Client sends `ClientKeyExchange` (contains 48-byte **Pre-Master Secret** encrypted with server's Public Key!).
• Client sends `CertificateVerify` (digitally signed hash proving client owns private key).

PHASE 4: Finish & Activate CipherSuite
• Client sends `ChangeCipherSpec` followed by `Finished` (encrypted verification hash).
• Server sends `ChangeCipherSpec` followed by `Finished` (encrypted verification hash).
• Secure channel established! Application data transfer begins.

4.5 SSL-Based Services & Standard Port Numbers

When application protocols are wrapped inside SSL/TLS, they run on designated secure port numbers:

Service / Protocol	Plaintext Port	SSL/TLS Encrypted Port	Description
HTTPS (HTTP over SSL)	Port 80	**Port 443**	Secure Web Browsing & E-commerce
LDAPS (LDAP over SSL)	Port 389	**Port 646**	Secure Directory Access & Authentication
SMTPS (SMTP over SSL)	Port 25	**Port 463** / 587	Secure Email Submission & Relay
POP3S (POP3 over SSL)	Port 110	**Port 995**	Secure Email Retrieval from Mail Server

MODULE 5: LECTURE 35 — APPLICATIONS (PART II): TLS, SSH, IPSEC & HTTPS

5.1 Transport Layer Security (TLS) & Secure Shell (SSH)

TLS (Transport Layer Security): Standardized by IETF as the modern successor to Netscape SSL. Operates at the transport layer to provide end-to-end security for web applications.

SSH (Secure Shell): Developed in 1995 to replace insecure, plaintext remote management protocols like Telnet and rlogin. Runs on **TCP Port 22**.

Key Architectural Features of SSH:
• **Port Forwarding / Tunneling:** Encapsulates and encrypts arbitrary TCP traffic (even insecure protocols) inside an encrypted SSH connection.
• **Dual Key Management Structure:**
- *Long-Term Server Key (1024 or 2048-bit RSA):* Binds server identity and handles initial connection authentication.
- *Short-Term Encryption Key (768-bit RSA):* Automatically re-generated and **refreshed frequently (e.g., hourly)** for bulk data encryption. Provides **Forward Secrecy** so compromising one session key does not compromise past sessions!
• **Multiple Authentication Options:** Encrypted passwords, RSA key pair authentication (`~/.ssh/authorized_keys`), Challenge-Response, or smart cards.

5.2 IP Security (IPsec) Framework & Modes

What it is: IPsec is a suite of security protocols operating directly at **Layer 3 (Network Layer)**. It provides host-to-host or network-to-network security. Mandatory for IPv6 and optional for IPv4.

Three Core Security Services Provided:
1. **Source Authentication & Data Integrity:** Verifies packet source IP address and ensures payload/header were not modified.
2. **Confidentiality:** Encrypts IP payload or entire IP packet to prevent eavesdropping.
3. **Anti-Replay Protection:** Tracks packet sequence numbers to detect and drop retransmitted duplicate packets.

IPsec Modes of Operation — Transport Mode vs. Tunnel Mode:

Feature / Aspect	IPsec Transport Mode	IPsec Tunnel Mode
Protection Scope	Protects **only the transport layer payload** (Data + TCP/UDP header).	Protects the **entire original IP packet** (Original IP Header + Data).
Original IP Header	**Visible / Unencrypted.** Routers read original IP header for routing.	**Encrypted & Hidden.** Original packet is encapsulated inside a *New Outer IP Header*.
Typical Deployment	End-to-End host communication (e.g., Client to Server).	Network-to-Network Virtual Private Networks (VPNs) between gateway firewalls.
NAT Compatibility	Compatible with standard routing.	Historically conflicted with NAT (modifying inner headers broke integrity checks).

5.3 S-HTTP vs. HTTPS — Critical Distinction

Students frequently confuse S-HTTP and HTTPS on exams. They operate at completely different abstraction levels:

Comparison Metric	S-HTTP (Secure HTTP)	HTTPS (HTTP over SSL/TLS)
Security Level	**Message-Level Security.** Secures individual HTTP request/response messages.	**Channel-Level Security.** Secures the *entire communication pipe* between browser and server.
Protocol Layer	Application Layer extension of HTTP.	HTTP layered on top of Transport Layer Security (SSL/TLS).

Client Certificates	Does not typically require public certificates from client side.	Supports mutual authentication via X.509 client certificates.
Industry Standard	Niche / Rarely used today.	**Global Web Standard (Port 443).** Used for all secure web traffic.

MODULE 6: WEEK 7 REVISION & NPTEL EXAM PREPARATION PAGES

6.1 Quick Week 7 Core Revision Summary

Core Takeaways to Memorize:

• **Cryptographic Hash Functions:** One-way functions mapping arbitrary input M to fixed-length output h. Provide *Data Integrity*.
• **MDC vs. MAC:** MDC is unkeyed (integrity only). MAC is keyed with shared secret K (integrity AND sender authentication).
• **Core Properties:** Variable input, fixed output, efficiency, First Preimage Resistance (one-way), Second Preimage Resistance (weak collision), Collision Resistance (strong collision).
• **SHA-512:** Processes 1024-bit blocks across 80 rounds using fractional cube roots of prime numbers; outputs 512-bit digest.
• **HMAC Construction:** $HMAC(K, M) = H((K \oplus opad) || H((K \oplus ipad) || M))$. Eliminates length extension attacks.
• **Birthday Attack:** Collision complexity is $2^{n/2}$ for n-bit hash output. Requires $n \ge 128$ for 64-bit security.
• **Digital Signatures:** Sign with **Private Key (d)**, verify with **Public Key (e)**. Provides Auth, Integrity, & Non-Repudiation.
• **Digital Certificates & PKI:** X.509 v3 certificates bind public keys to identities using CAs.
• **SSL/TLS Architecture:** SSL Record Protocol (≤ 16 KB fragmentation) + Handshake, ChangeCipherSpec, Alert protocols.
• **IPsec Modes:** Transport Mode (protects payload only, original IP header visible) vs. Tunnel Mode (encapsulates entire IP packet inside new header; standard for VPNs).

6.2 Master Glossary & Core Definitions

- **Cryptographic Hash Function:** Mathematical function mapping arbitrary inputs to a fixed-length digest with one-way and collision-resistant properties.
- **Modification Detection Code (MDC):** An unkeyed hash value appended to a message to verify data integrity in transit.
- **Message Authentication Code (MAC):** A keyed hash value calculated using a shared secret key K to guarantee both integrity and sender origin.
- **First Preimage Resistance:** Property making it computationally infeasible to find input x given hash value $y = H(x)$.
- **Collision Resistance:** Property making it computationally infeasible to find any pair $x \neq x'$ such that $H(x) = H(x')$.
- **HMAC:** Hash-based Message Authentication Code construction using nested hashing with $ipad$ (`0x36`) and $opad$ (`0x5C`).
- **Birthday Attack:** A cryptographic attack exploiting probability theory to find hash collisions in $2^{n/2}$ attempts.
- **Digital Signature:** A cryptographic scheme where a sender encrypts a message digest with their private key, providing non-repudiation.
- **X.509 Certificate:** An international standard electronic document issued by a CA that binds a public key to an entity.
- **IPsec Tunnel Mode:** An IPsec mode that encapsulates and encrypts the entire original IP packet inside a new IPsec outer header.

6.3 Master Formula & Equation Sheet

FORMULA & EQUATION SHEET FOR WEEK 7

1. **SHA-512 Message Padding Condition:**
   $(L + 1 + k + 128) \equiv 0 \pmod{1024}$ (where L = message length in bits, k = zero-padding bits)

2. **HMAC Mathematical Construction:**
   $HMAC(K, M) = H\Big((K \oplus opad) \;||\; H\big((K \oplus ipad) \;||\; M\big)\Big)$

3. **Birthday Attack Collision Complexity:**
   $\text{Collision Search Attempts} \approx 2^{n/2}$ (for an n-bit hash output digest)

4. **Digital Signature Generation & Verification:**
   $\text{Signing (Alice): } S = f_s(H(M), d_A)$ $\text{Verifying (Bob): } m'' = f_v(S, e_A) \stackrel{?}{==} H(M')$

6.4 Important Solved Numerical Examples

SOLVED EXAMPLE 1 (SHA-512 Padding Bit Calculation):
Problem: A message M has a length of $L = 2000$ bits. Calculate the number of zero padding bits k required for SHA-512.
Solution Step-by-Step:
1. SHA-512 processes 1024-bit blocks. Condition: $(2000 + 1 + k + 128) \equiv 0 \pmod{1024}$.
2. Combine constants: $2129 + k \equiv 0 \pmod{1024}$.
3. Next multiple of 1024 above 2129 is $3 \times 1024 = 3072$.
4. Solve for k: $k = 3072 - 2129 = \mathbf{943\text{ zero bits}}$.

SOLVED EXAMPLE 2 (Birthday Attack Collision Attempts):
Problem: An old legacy system uses a 64-bit hash digest. How many attempts are required for an attacker to find a collision using a Birthday Attack?
Solution Step-by-Step:
1. Hash digest size $n = 64$ bits.
2. Birthday attack complexity formula $= 2^{n/2} = 2^{64/2} = 2^{32}$.
3. Answer: Attacker requires only $\mathbf{2^{32} \approx 4.29 \times 10^9\text{ attempts}}$, which takes seconds on modern hardware!

6.5 NPTEL Exam Practice Questions (Incorporating Assignment 7)

Q1: A message M is fed to a hash function to generate hash value H = HASH(M). Which statement(s) is/are true?

a. Number of bits in H is typically much smaller than M.
b. Output length of H grows proportionally with input length.
c. Hashing produces a fixed-length output regardless of input size.
d. H and M are always equal in length.

Correct Answer: a, c | *Explanation:* A hash function compresses an arbitrarily large input message into a fixed-length digest regardless of input size.

Q2: Which of the following statement(s) is/are true regarding cryptographic algorithms?

a. The block size of DES is 64 bits.
b. The key size of DES is 128 bits.
c. The hash digest size of MD5 is 128 bits.
d. SHA-256 produces a 512-bit digest.

Correct Answer: a, c | *Explanation:* DES uses a 64-bit block size (56-bit key). MD5 produces a 128-bit hash digest. SHA-256 produces 256 bits.

Q3: Which of the following statement(s) is/are true regarding SSL/TLS protocols?

a. HTTPS relies on SSL/TLS to secure HTTP communication.
b. SSL/TLS guarantees faster data transmission than plain HTTP.
c. TLS operates at the physical layer of the network stack.
d. SSL Record Protocol guarantees non-repudiation.

Correct Answer: a | *Explanation:* HTTPS runs HTTP over SSL/TLS above the IP layer to secure communications.

Q4: Which statement is true regarding collision resistance in hash functions?

a. It is a weaker requirement than second preimage resistance.
b. It should be computationally difficult to find any two distinct messages producing the same hash.
c. Collision resistance guarantees encryption confidentiality.
d. Collision resistance applies only to keyed hash functions.

Correct Answer: b | *Explanation:* Collision resistance requires that finding any two distinct messages $x \neq x'$ with $H(x) = H(x')$ is computationally infeasible.

Q5: Which statement(s) is/are true regarding digital signature verification?

a. The verifier uses the signer's public key to verify a signature.
b. The verifier uses the signer's private key.
c. Successful verification confirms message was not altered and signed by claimed signer.
d. Verification requires signer to be online.

Correct Answer: a, c | *Explanation:* Verification uses the signer's public key and confirms message integrity and sender origin.

Q6: Which property of digital signatures ensures that the original signer cannot later deny having signed a message?

a. Confidentiality | b. Integrity | c. Non-repudiation | d. Availability

Correct Answer: c | *Explanation:* Non-repudiation ensures a signer cannot deny having created the digital signature.

Q7: Which statement is FALSE regarding keyed vs. unkeyed hash functions?

a. Unkeyed hash function can verify integrity if hash comes from trusted source.
b. Keyed hash function requires a secret key.
c. An unkeyed hash function inherently provides authenticity.
d. HMAC provides integrity and sender authentication.

Correct Answer: c | *Explanation:* Unkeyed hash functions (MDCs) provide data integrity ONLY, not authenticity.

Q8: In IPSec, which mode encrypts/protects only the payload while keeping the original IP header intact?

a. Tunnel mode | b. Transport mode | c. Replay mode | d. Confidential mode

Correct Answer: b | *Explanation:* Transport mode protects transport layer payload while keeping the original IP header visible.

Q9: What is the exact hash digest size produced by the MD5 algorithm?

a. 64 bits | b. 128 bits | c. 160 bits | d. 256 bits

Correct Answer: b | *Explanation:* MD5 produces a fixed-length output digest of exactly 128 bits.

Q10: The block size processed in each round by the SHA-512 hashing algorithm is:

a. 512 bits | b. 80 bits | c. 1024 bits | d. 2048 bits

Correct Answer: c | *Explanation:* SHA-512 processes input messages in 1024-bit blocks to produce a 512-bit digest.

6.6 Oral Viva Questions & Easy Answers

Q: 1. What is the difference between MDC and MAC?
A: MDC is unkeyed (provides integrity only). MAC uses a secret key K (provides integrity AND sender authentication).

Q: 2. What is First Preimage Resistance?
A: The one-way property making it infeasible to reverse a hash $y = H(x)$ back to original message x.

Q: 3. What is the Birthday Attack?
A: A collision attack requiring $2^{n/2}$ attempts for an n-bit hash function based on probability theory.

Q: 4. How does a Digital Signature work?
A: Sender encrypts message hash using private key (d). Receiver decrypts signature using sender's public key (e) and compares hashes.

Q: 5. What is the role of a Certification Authority (CA)?
A: A trusted third party that issues X.509 digital certificates binding an entity's identity to their public key.

Q: 6. What happens in Phase 3 of the SSL Handshake?
A: Client sends `ClientKeyExchange` containing the Pre-Master Secret encrypted with the server's public key.

Q: 7. What is the function of the SSL ChangeCipherSpec protocol?
A: Single 1-byte message (`0x01`) signaling immediate activation of newly negotiated security parameters.

Q: 8. Compare IPsec Transport Mode and Tunnel Mode.
A: Transport mode protects payload only (original IP header visible). Tunnel mode encrypts entire original IP packet inside a new IPsec header.

Q: 9. What is the difference between S-HTTP and HTTPS?
A: S-HTTP secures individual HTTP messages. HTTPS secures the entire communication channel by running HTTP over SSL/TLS.

Q: 10. Why is SSH considered superior to Telnet?
A: Telnet transmits credentials in plaintext. SSH encrypts all communications and provides secure authentication and tunneling.

6.7 Final Revision Checklist

Mark checkboxes as you master each topic before the NPTEL exam:

[ ] Understand Cryptographic Hash Functions, MDCs (unkeyed), and MACs (keyed).

[ ] Memorize 6 Core Security Properties of Hash Functions (1st/2nd Preimage & Collision Resistance).

[ ] Master SHA-512 padding bit calculation $(L+1+k+128) \equiv 0 \pmod{1024}$ and 80-round structure.

[ ] Explain HMAC construction formula $HMAC(K, M)$ and why it prevents length extension attacks.

[ ] Calculate Birthday Attack collision attempts $2^{n/2}$ and minimum output size requirements.

[ ] Understand Digital Signatures (Private Key signing, Public Key verification) & Non-Repudiation.

[ ] Learn X.509 v3 Digital Certificate fields, CAs, and Certificate Revocation Lists (CRLs).

[ ] Explain 2-Layer SSL Architecture, SSL Record Protocol pipeline (≤ 16 KB fragmentation), and 4-Phase Handshake.

[ ] Memorize SSL/TLS ports: HTTPS (443), LDAPS (646), SMTPS (463), POP3S (995).

[ ] Differentiate IPsec Transport vs. Tunnel Mode, SSH key management (long-term vs short-term keys), and S-HTTP vs. HTTPS.